

Player Health & Stats
Types of Enemies

Prefabs

- A prefab is a master entity of a GameObject that you can use in your game endlessly. In other words, creating a prefab is like creating a template that contains all the characteristics of something (e.g. Enemy), and that template can be used to create many copies. That saves a lot of time.

To Create a Prefab

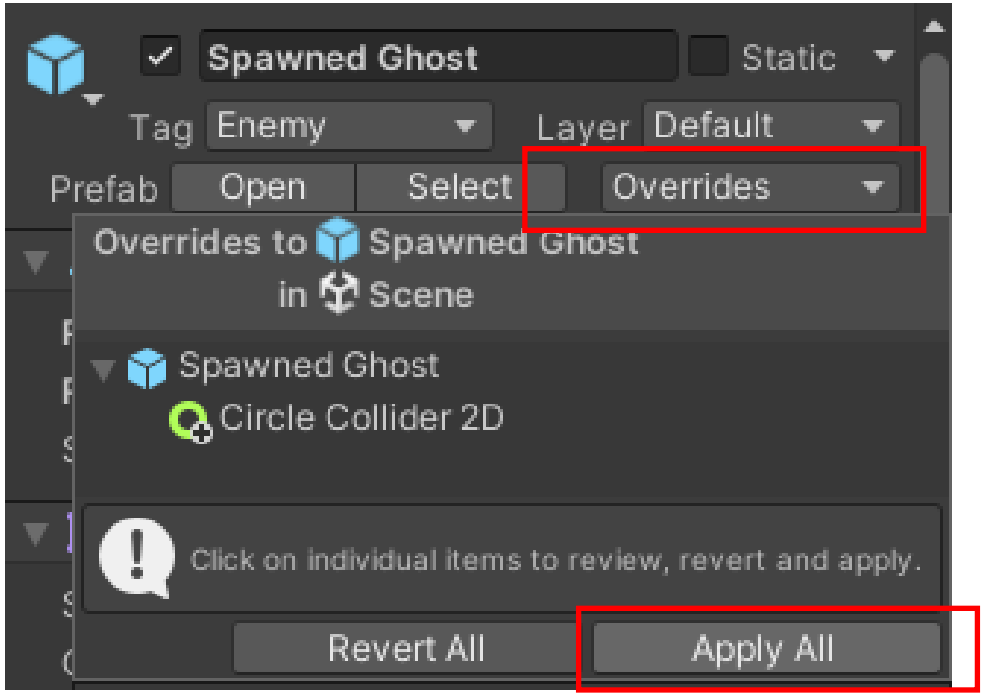
- To create a prefab, first create a folder in the Project window under Assets and name it Prefabs

- Select the GameObject you want to turn into a prefab from the Hierarchy window and drag and drop it to the prefab you created

- If it turns blue, then it has been created successfully

To Update a Prefab

- If you ever want to make a change to a prefab and want it to reflect on ALL the copies in the game (e.g. you made a change to a Ghost Enemy and you want all instances of the Ghost Enemy to be updated), you must go to the top of the Hierarchy Window, click the Overrides drop down list and click Apply All.



Exercise #18 – Player Stats

- Create a script that contains everything needed to control the player's health and lives. Call it PlayerStats and attach it to the Player gameObject

- We need the player stats to have 3 things:
  - Health
  - Lives
  - Temporary Immunity

- Variables we'll need are listed on the right:

```
public int health = 3;
public int lives = 3;

private float flickerTime = 0f;
public float flickerDuration = 0.1f;

private SpriteRenderer sr;

public bool isImmune = false;
private float immunityTime = 0f;
public float immunityDuration = 1.5f;
```

Exercise #18 – Player Stats (Cont.)

- In the Start() function:

```
void Start()
{
    sr = GetComponent<SpriteRenderer>();
}
```

- Create a SpriteFlicker() function

```
void SpriteFlicker()
{
    if(flickerTime < flickerDuration)
    {
        flickerTime = flickerTime + Time.deltaTime;
    }
    else if (flickerTime >= flickerDuration)
    {
        sr.enabled = !(sr.enabled);
        flickerTime = 0;
    }
}
```

Exercise #18 – Player Stats (Cont.)

- Create the TakeDamage() function

```
public void TakeDamage(int damage)
{
    if (isImmune == false )
    {
        health = health - damage;
        if (health < 0)
            health = 0;
        if (lives > 0 && health == 0)
        {
            FindObjectOfType<LevelManager>().RespawnPlayer();
            health = 3;
            lives--;
        }
        else if (lives == 0 && health == 0)
        {
            Debug.Log("Gameover");
            Destroy(this.gameObject);
        }

        Debug.Log("Player Health:" + health.ToString());
        Debug.Log("Player Lives:" + lives.ToString());
    }

    isImmune = true;
    immunityTime = 0f;
}
```

Exercise #18 – Player Stats (Cont.)

- In the Update() function:

```
void Update()
{
    if(isImmune == true)
    {
        SpriteFlicker();
        immunityTime = immunityTime + Time.deltaTime;
        if(immunityTime >= immunityDuration)
        {
            isImmune = false;
            sr.enabled = true;
        }
    }
}
```

Enemies

- You will need to add enemies to your levels to make them more interesting

- Examples we'll go through are:
  - Walk on the ground
  - Spawn when player passes a specific point
  - Follow the player

Exercise #19 - Create A Super Class For Your Enemies

- To avoid redundancy in the code, create a script and name it EnemyController
- This script will include functions and variables that can be inherited by various types of enemies. Add those 3:

```
public float maxSpeed = 2;
public int damage = 1;
public SpriteRenderer sr;
```

  - (Note that since the variables are public anyway, you don't need to give them specific values inside the code. Do it in Unity instead)

Exercise #19 - Create A Super Class For Your Enemies (Cont.)

- Also create a Flip() function that allows your enemy to flip when it hits a wall or another enemy.

```
void Start()
{
    sr = GetComponent<SpriteRenderer>();
}
//If true turn to false, if false turn to true
public void Flip()
{
    sr.flipX =!sr.flipX;
}
```

Exercise #19 - Create A Super Class For Your Enemies (Cont.)

- Whenever the player runs into the enemy's collider (which has the "IsTrigger" property checked), the function TakeDamage() is called from the PlayerStats script, and the player's health receives damage. Then the enemy flips and moves away

```
void OnTriggerEnter2D(Collider2D other)
{
    if (other.tag == "Player")
    {
        FindObjectOfType<PlayerStats>().TakeDamage(damage);
        Flip();
    }
    else if (other.tag == "Wall")
    {
        Flip();
    }
```

- Also, upon hitting a wall, the enemy also flips and moves away (remember: the walls must be tagged 'Wall' in Unity)

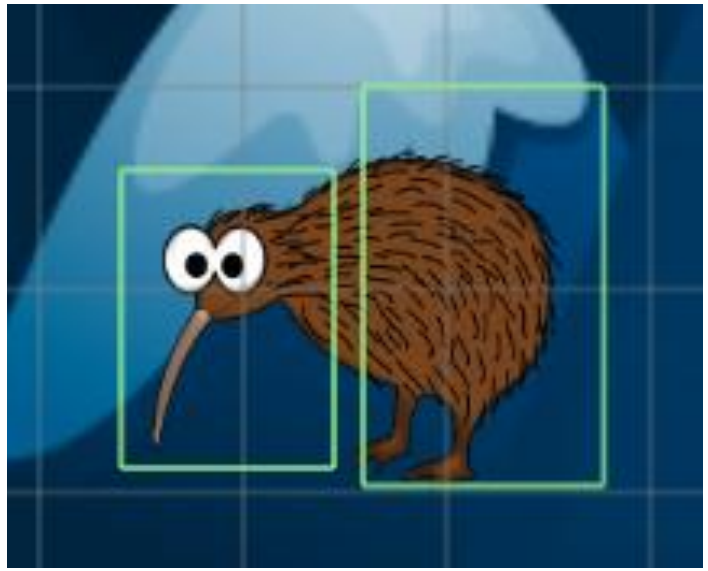


Exercise #20 - Create a normal walking enemy

- Choose an enemy sprite, and add a Rigidbody2D Component to it. Check the Z rotation freezing checkbox so the enemy can’t get flipped over if

- Add a normal box Collider 2D Component to the enemy’s body

- Add a triggered Box Collider 2D Component to the enemy’s head. The trigger will be used to check for collisions with walls or the player, which would cause the enemy to turn around and move in the opposite direction



Exercise #20 - Create a normal walking enemy (Cont.)

- Create a script for your enemy gameObject and name it something relevant like WalkingEnemy, or SlimeEnemy, etc.

- The FixedUpdate() function is to make the character begin moving on the x-axis as soon as gameplay begins, with the sprite looking in the correct direction.

- See next slide for the script:

Exercise #20 - Create a normal walking enemy (Cont.)

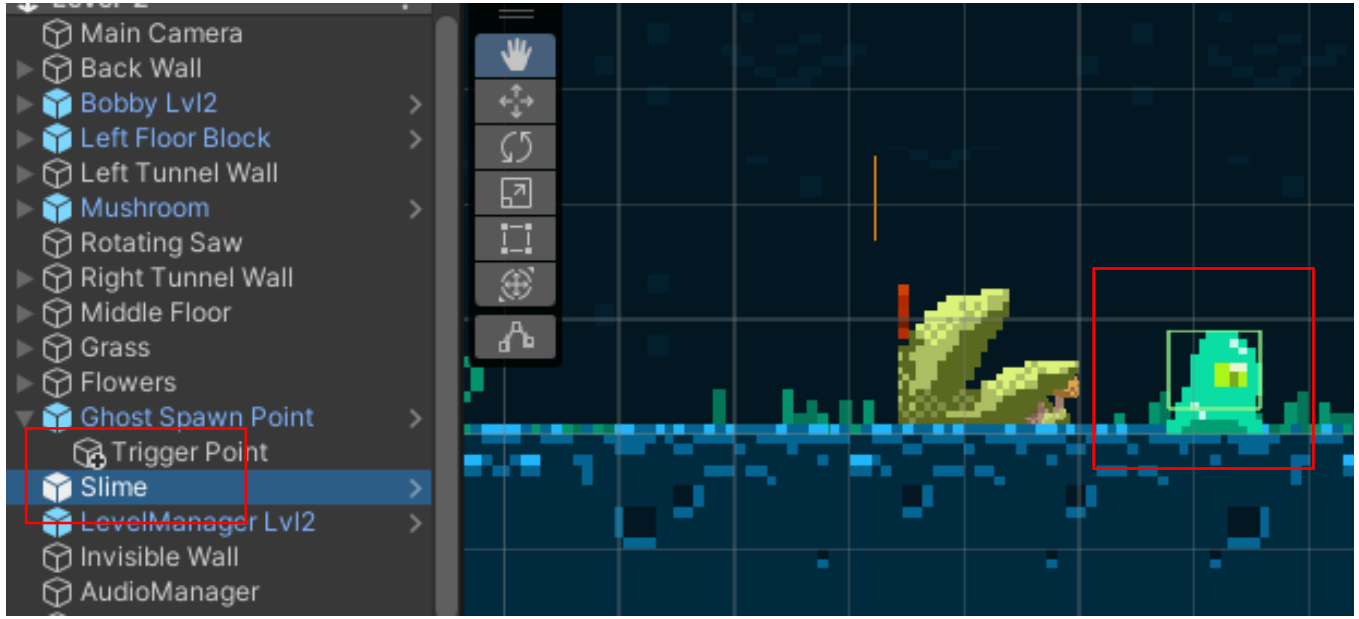
- Don’t forget to inherit from the Super class.

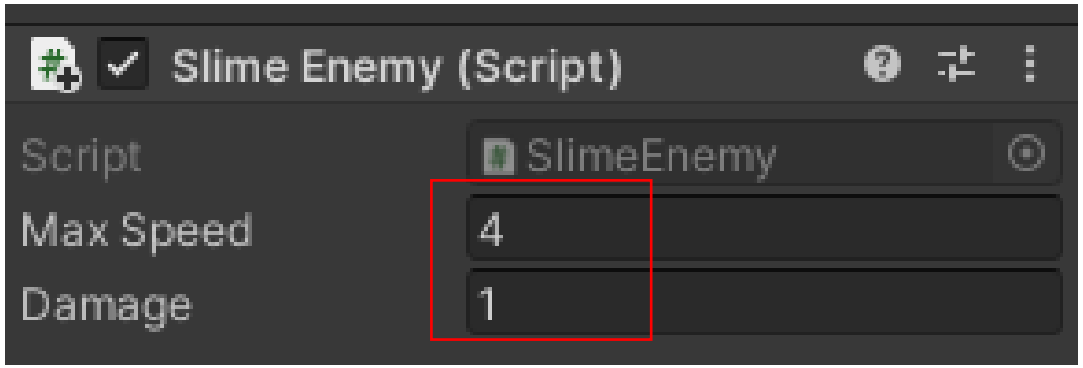
```
public class SlimeEnemy : EnemyController
{
    // Start is called before the first frame update
    void Start()
    {
        sr = GetComponent<SpriteRenderer>();
        sr.flipX = false;
    }

    void FixedUpdate ()
    {
        //if(this.isFacingRight == true)
        if(sr.flipX==true)
        {
            this.GetComponent<Rigidbody2D>().velocity =
                new Vector2(-maxSpeed, this.GetComponent<Rigidbody2D>().velocity.y);
        }
        else
        {
            this.GetComponent<Rigidbody2D>().velocity =
                new Vector2(maxSpeed, this.GetComponent<Rigidbody2D>().velocity.y);
        }
    }
```

Exercise #20 - Create a normal walking enemy (Cont.)

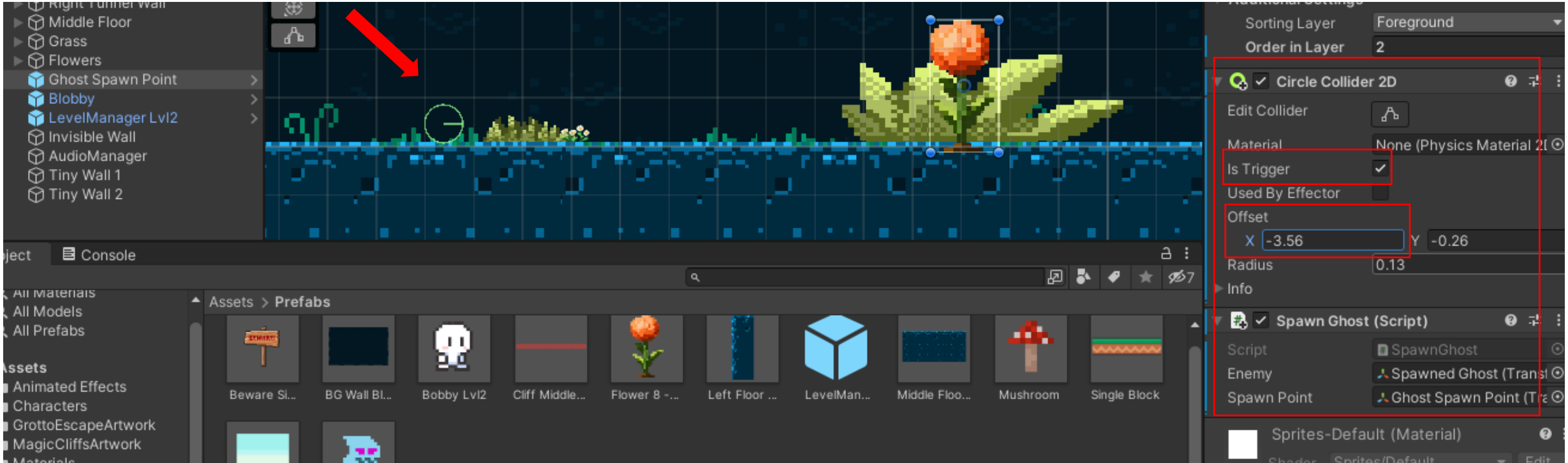
- Back in the Unity interface, don't forget to give the enemy's public variables values





Exercise #21 - Create an Enemy that Spawns at a Certain Point

- Create an empty gameObject and add a triggered collider2D and a script file to it. Name it SpawnPoint.
  - It is named so because the enemy appears only when the player **runs through the gameObject's collider.** Offset the collider to the left.



Exercise #21 - Create an Enemy that Spawns at a Certain Point (Cont.)

- We want to make an enemy appear the moment the player passes the triggered collider. We'll need to write 2 scripts: one for the spawning process, and a Ghost enemy
- Name them SpawnGhost and GhostEnemy respectively
- Like the previous enemy, this one also inherits from EnemyController

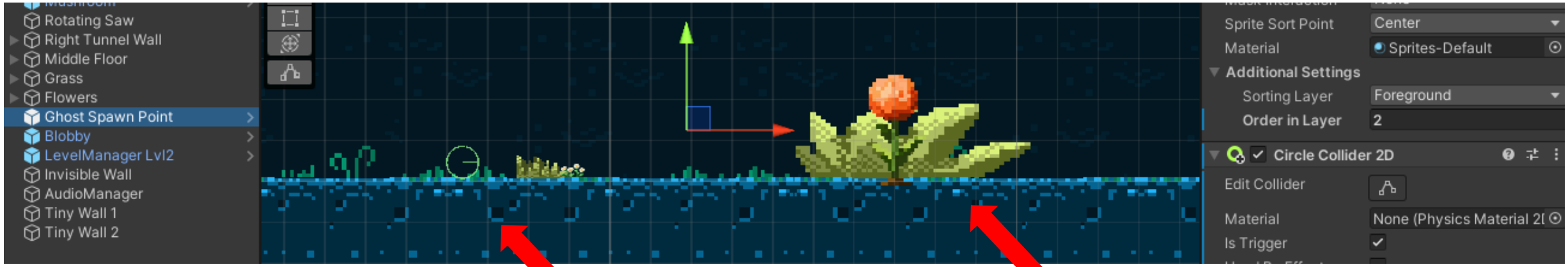


- Create a ghost enemy prefab and keep it ready in the prefabs folder
- The script for GhostEnemy will override the OnTriggerEnter2D function() because ghosts don't collide with walls:

```
//Because ghosts can pass through walls, the Trigger function is overridden from the Superclass
//to remove the flip function upon hitting a wall
void OnTriggerEnter2D(Collider2D other)
{
    if (other.tag == "Player")
    {
        FindObjectOfType<PlayerStats>().TakeDamage(damage);
        Flip();
    }
```

Exercise #21 - Create an Enemy that Spawns at a Certain Point (Cont.)

- Don’t forget to decide where you want the trigger point to be, and where the ghost will appear from:



Trigger point

Spawn point

Exercise #21 - Create an Enemy that Spawns at a Certain Point (Cont.)

- Now add this script to the trigger point
- We need 2 variables: one to store the ghost prefab, the other to store the flower that serves as the spawn point

```
//Place the respawnable enemy prefab into this variable
public Transform enemy;

//Place the gameObject you want the enemy prefab to spawn at
public Transform spawnPoint;
```

Exercise #21 - Create an Enemy that Spawns at a Certain Point (Cont.)

- Now create a RespawnEnemy() function:

```
void RespawnEnemy()
{
    //Create an instance of the enemy at the spawn point's position and rotation values
    Instantiate(enemy, spawnPoint.transform.position, spawnPoint.transform.rotation);
}

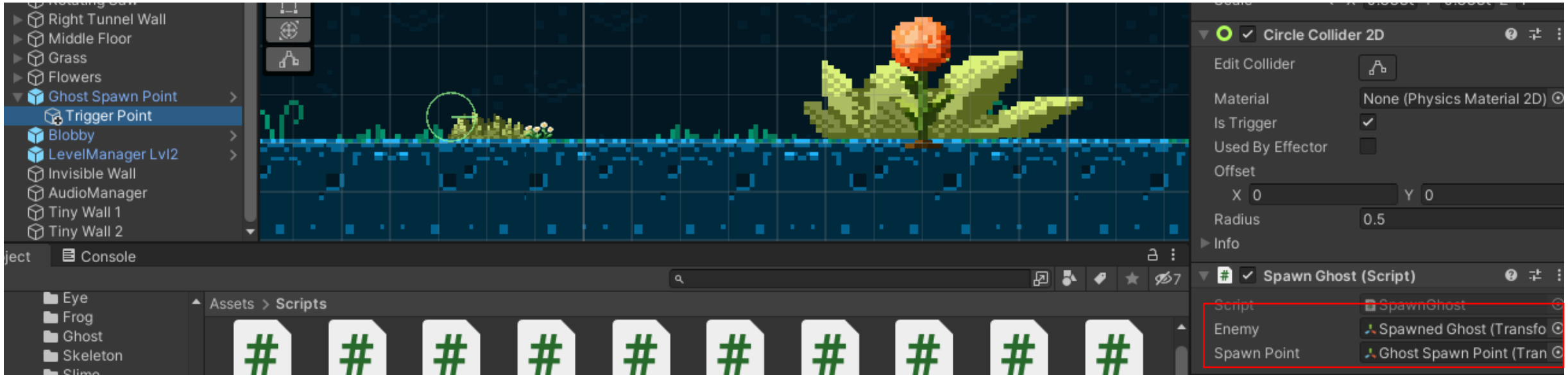
void OnTriggerEnter2D(Collider2D other)
{
    if (other.gameObject.tag == "Player")
    {
        RespawnEnemy();
    }
}
```

- Instantiate() is a built-in function in Unity that allows you to create an instance of the prefab you chose (in this case, the Ghost)

- Finally, the OnTriggerEnter2D() function will activate upon Player passing through, and call on the ghost to respawn

Exercise #21 - Create an Enemy that Spawns at a Certain Point (Cont.)

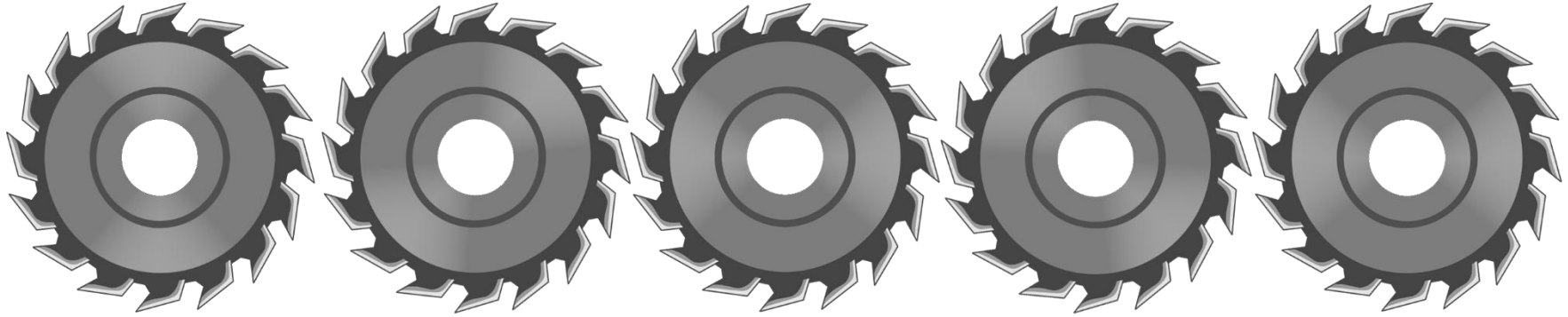
- Finally, go back to Unity and make sure you drag and drop the enemy prefab and spawn point you have created into the boxes.



Exercise #22 – Create an enemy that follows the player

- Create another prefab of an enemy gameObject, and create a script for it called FollowerEnemy. Wherever your player goes, this enemy follows them.
- FollowerEnemy is also a subclass of EnemyController.

- A good example of a FollowerEnemy is a rotating saw like this. It can be animated and follows the player left and right



Exercise #22 – Create an enemy that follows the player (Cont.)

- The *Transform* variable will be used to refer to the player's object location

```
public class MovingSaw : EnemyController
{
    public Transform player;
```

- In FixedUpdate(), call the predefined function MoveTowards() to make the enemy follow the player wherever he goes

```
void FixedUpdate()
{
    transform.position = Vector3.MoveTowards(transform.position, player.transform.position,
    maxSpeed * Time.deltaTime);
}
```

Exercise #22 – Create an enemy that follows the player (Cont.)

- And finally, override the OnTrigger() function and delete the flip function call since we do *not* want the saw to flip and turn away if it hits the player, only the wall.
- Don’t forget to go back and fill in the values in Unity

```
//Since the saw is not supposed to flip if it collides with player, the Trigger function will be
//overridden as follows:
void OnTriggerEnter2D(Collider2D other)
{
    if (other.tag == "Player")
    {
        FindObjectOfType<PlayerStats>().TakeDamage(damage);
    }
    else if (other.tag == "Wall")
    {
        Flip();
    }
}
```

